Hospital Appointment System

Domain Scenario:

A patient wants to book an appointment with a doctor in a specific department. The system must check the doctor's available time slots, confirm the appointment, generate a token, and create a billing record. Later, the doctor adds medical notes and a prescription to the patient's medical record.

1. Possible Conceptual Classes and Their Purposes (Using Noun-Phrase + Category List Technique)

No.	Class Name	Larman Category	Purpose / Justification
1	Patient	Role of Person / Physical	The primary actor who seeks medical consultation. The system exists for them.
2	Doctor	Role of Perso	Medical professional who provides consultation. Has specialization and schedule.
3	Appointment	Transaction / Event	The CORE transaction of the system. Links Patient, Doctor, and Time Slot together.
4	Department	Container / Organization	Groups doctors by specialty, e.g., Cardiology, Neurology. Helps patients find the right doctor.
5	Schedule / TimeSlot	Time Period / Event	Represents doctor's availability. Essential to check if a doctor is free.
6	MedicalRecord	Record / Specification	Holds a patient's history, past appointments, diagnoses, allergies. Persistent record.
7	Prescription	Transaction / Specification	Written by doctor after appointment. Describes medicines and dosage.
8	Billing / Payment	Transaction	Handles consultation fee, payment status, and invoice generation.
9	Hospital	Organization / Place	The overall container that owns departments, doctors, and rooms.
10	Room / Clinic	Place / Tangible	Physical place where the appointment happens. Has room number and capacity.
11	Receptionist / Admin	Role of Person	Manages appointments, verifies patients, and supports system operations.
12	Notification	Event (Invented)	Sends SMS/email reminders about appointment time, token number, or cancellation.

Filtered out attributes like "Name", "PhoneNumber", "Date" - they are not classes, they are attributes of Patient/Doctor.

2. Three Most Important Classes- Detailed Attributes & Methods

"This follows your slide "State, Behavior, Identity"

1. **Patient**

Attributes: patientID, name, dateOfBirth, gender, contactNumber, email, address, medicalHistorySummary, registeredAppointments

Methods: register(), login(), bookAppointment(doctor, slot), cancelAppointment(appointmentID), viewMedicalHistory(), makePayment(), updateProfile()

2. **Doctor**

Attributes: doctorID, name, specialization, department, qualification, contactNumber, consultationFee, availabilitySlots, currentAppointments, rating

Methods: viewTodayAppointments(), updateAvailability(), isAvailable(date, time), addMedicalNotes(), writePrescription(), viewPatientRecord()

3. **Appointment**

Attributes: appointmentID, patient, doctor, department, appointmentDate, timeSlot, tokenNumber, status, reasonForVisit, room

Methods: scheduleAppointment(), reschedule(newSlot), cancel(reason), generateToken(), calculateWaitTime(), sendReminder(), checkAvailability()

4. **Hospital**

Attributes: hospitalID, name, address, contact, listOfDepartments, listOfDoctors

Methods: addDepartment(), getDepartmentInfo(), getAllDoctors()

3. Abstraction Decisions

Abstraction is the art of leaving things out. For the Hospital Appointment domain, we deliberately kept only what is relevant to booking and consulting. We IGNORED the following real-world details: The patient's favorite food, height, weight, blood group details beyond medical need, the doctor's personal family life, hospital's cafeteria menu, parking management, detailed inventory of medicines in pharmacy, staff payroll and salaries, and the cleaning staff management. We also ignored low-level technical details like UI screens, database tables, and login password encryption. Why? Because our domain focus is Appointment Booking, not Hospital Management as a whole. If we include everything, we would create a God Class / Blob - a mistake mentioned in your slides. We kept our classes

purposeful and simple, just like the student class in Lecture 02 only had studentID, name, email, not blood type and shoe size.

4. Invented Software Object

Not every useful object exists in the real world. Our invented object is: AppointmentScheduler / AppointmentValidator (Controller + Service) Purpose: This object does not exist as a person in a real hospital. In real life, a receptionist does this manually. But in software, we need a central controller to manage the complex logic.It will:Orchestrate the booking flow: When Patient calls bookAppointment(), the Scheduler checks Doctor.isAvailable(), checks if Patient has another appointment at same time, generates tokenNumber, creates Appointment and Billing objects, and triggers Notification.Prevent double-booking and validate rules (e.g., a doctor cannot have more than 20 appointments per day).Manage the waitlist if a slot is full.This follows the slide "Software Invents Objects Too" - Controllers, Validators, and Services.